

Programación Científica · 2026-1

Redes Neuronales

Otra vez: Aproximar funciones

Universidad Nacional de Colombia

`mbastidaso@unal.edu.co`

La pregunta de hoy:

¿Cómo aproximamos
funciones muy complicadas?

(dejen esta pregunta abierta — la responderemos al final)

Parte 1

El hilo del curso

El hilo del curso

El mismo problema, con más libertad

En todos los métodos anteriores buscamos:

$$\hat{f}(x) = \sum_{i=1}^n c_i \phi_i(x)$$

Las bases ϕ_i estaban **fijas**. En una red neuronal, los ϕ_i **también dependen de parámetros** que se optimizan.

¿Qué ganamos con bases aprendibles?

Bases fijas

- ▶ Escogemos la forma de ϕ_i antes de ver los datos
- ▶ Funciona bien si tenemos intuición sobre la estructura
- ▶ Monomios: oscilación de Runge
- ▶ Splines: necesitamos nodos bien ubicados

Bases aprendibles

- ▶ Los ϕ_i se adaptan a los datos
- ▶ No necesitamos saber la estructura a priori
- ▶ El precio: necesitamos **optimizar** los parámetros
- ▶ ¿Cómo? $\rightarrow$ gradiente

Hoy: construimos la maquinaria mínima para entender cómo se optimiza.

Parte 2

La red neuronal mínima

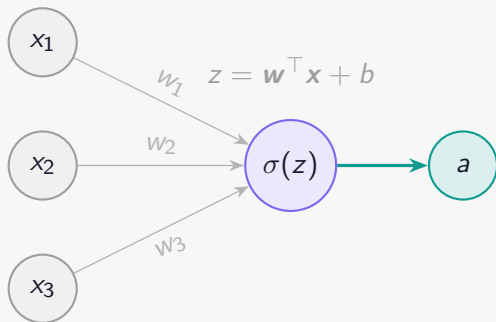
Una neurona

Neurona (ML)

Recibe entradas $x_1, \dots, x_n$, calcula:

$$z = \sum_{j=1}^n w_j x_j + b, \quad a = \sigma(z)$$

- ▶ w_j : pesos (*weights*)
- ▶ b : sesgo (*bias*)
- ▶ σ : función de activación (no lineal)



Activaciones comunes:

$$\sigma(z) = \tanh(z)$$

$$\sigma(z) = \max(0, z) \text{ (ReLU)}$$

$$\sigma(z) = (1 + e^{-z})^{-1} \text{ (sigmoid)}$$

Red feedforward de una capa oculta

Arquitectura

$$\hat{f}(x; \theta) = w^2 \sigma(W^1 x + \mathbf{b}^1) + b_1^2$$

- ▶ Capa oculta: $\mathbf{h} = \sigma(W_1 x + \mathbf{b}_1) \in \mathbb{R}^m$
- ▶ Capa de salida: $\hat{f} = W_2 \mathbf{h} + b_2 \in \mathbb{R}$
- ▶ Parámetros: $\theta = \{W_1, \mathbf{b}_1, W_2, b_2\}$

Dimensiones

Si $x \in \mathbb{R}$ y tengo m neuronas ocultas:

$W_1 \in \mathbb{R}^{m \times 1}$, $\mathbf{b}_1 \in \mathbb{R}^m$, $W_2 \in \mathbb{R}^{1 \times m}$, $b_2 \in \mathbb{R}$

Total: $3m + 1$ parámetros.

Dibujemos un grafo

Ejercicio

Considere la red con $m = 2$ neuronas ocultas y entrada escalar x :

$$h_1 = \sigma(w_1^1 x + b_1^1), \quad h_2 = \sigma(w_2^1 x + b_2^1), \quad \hat{f} = w_1^2 h_1 + w_2^2 h_2 + b_1^2$$

Dibujen el grafo computacional.

¿Cuántos parámetros tiene esta red?

Variables intermedias

$$z_1 = w_1^1 x + b_1$$

$$h_1 = \sigma(z_1)$$

$$z_2 = w_2^1 x + b_2$$

$$h_2 = \sigma(z_2)$$

$$\hat{f} = w_1^2 h_1 + w_2^2 h_2 + b_1^2$$

Parte 3

La función de pérdida

¿Cómo medimos el error?

Queremos que $\hat{f}(x; \theta) \approx f(x)$ para todo $x \in [a, b]$.

Loss function (pérdida L2)

$$\mathcal{L}(\theta) = \int_a^b (f(x) - \hat{f}(x; \theta))^2 dx$$

En la práctica, aproximamos la integral con N puntos de muestra:

Pérdida discreta (MSE)

$$\mathcal{L}(\theta) \approx \frac{1}{N} \sum_{i=1}^N (f(x_i) - \hat{f}(x_i; \theta))^2$$

Los $\{x_i\}$ se llaman **datos de entrenamiento**.



Teoría

¿cómo optimizamos?

La pérdida MSE es un funcional cuadrático

Sea $\hat{f}(x; \theta) = \theta^\top \phi(x)$ una red **lineal** en θ (e.g. regresión lineal, red sin activación). Con N datos $\{(x_i, y_i)\}$:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \theta^\top \vec{\phi}(x_i))^2 = \frac{1}{N} \|y - A\theta\|^2$$

donde $y = (y_1, \dots, y_N)^\top$ y $A_{ij} = \phi_j(x_i)$.

Conexión con mínimos cuadrados

$\nabla_{\theta} \mathcal{L} = 0$ da las ecuaciones normales $Q\theta^* = \ell$, exactamente el sistema de mínimos cuadrados que ya conocen. El mínimo global existe y es único si A tiene rango completo.

Gradiente de la pérdida cuadrática

Gradiente explícito (caso lineal)

$$\nabla_{\theta} \mathcal{L}(\theta) = \frac{2}{N} A^{\top} (A\theta - y) = \frac{2}{N} A^{\top} (\hat{y} - y)$$

donde $\hat{y} = A\theta$ son las predicciones actuales.

¿Por qué el gradiente apunta al mínimo?

Para cualquier θ , el vector $-\nabla_{\theta} \mathcal{L}(\theta)$ apunta en la dirección de **mayor descenso local** de $\mathcal{L}$. Para una forma cuadrática convexa, este descenso es global.

Generalización

Redes no lineales: $\hat{f}(x; \theta)$ arbitraria

Aproximación cuadrática local (caso no lineal)

Para una red no lineal, $\mathcal{L}(\theta)$ puede ser no convexa. Pero localmente, alrededor de θ_k , aproximamos por Taylor de orden 2:

$$\mathcal{L}(\theta_k + \delta) \approx \mathcal{L}(\theta_k) + \nabla_{\theta} \mathcal{L}(\theta_k)^{\top} \delta + \frac{1}{2} \delta^{\top} H_k \delta,$$

donde $H_k = \nabla_{\theta}^2 \mathcal{L}(\theta_k)$ es la **matriz Hessiana**.

Mínimo local de la aproximación cuadrática

Si $H_k \succ 0$, el mínimo de la aproximación es:

$$\delta^* = -H_k^{-1} \nabla_{\theta} \mathcal{L}(\theta_k).$$

Gradiente descendente usa $\delta = -\eta \nabla_{\theta} \mathcal{L}$, es decir, aproxima $H_k \approx \frac{1}{\eta} I$.

Convergencia en el caso general

Función L -suave

$\mathcal{L}$ es tal que $\|\nabla\mathcal{L}(\theta) - \nabla\mathcal{L}(\theta')\| \leq L\|\theta - \theta'\|$ para todo θ, θ' , entonces $\nabla^2\mathcal{L}(\theta) \preceq LI$.

Teorema (descenso suficiente, caso L -suave)

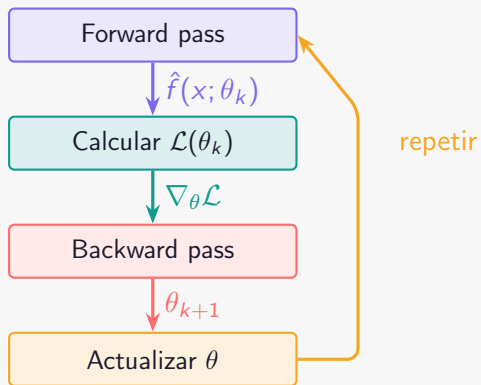
Si $\mathcal{L}$ es L -suave y $\eta \leq 1/L$, entonces:

$$\mathcal{L}(\theta_{k+1}) \leq \mathcal{L}(\theta_k) - \frac{\eta}{2} \|\nabla_{\theta}\mathcal{L}(\theta_k)\|^2.$$

Consecuencia

$\mathcal{L}(\theta_k) \rightarrow$ mínimo local si $\nabla\mathcal{L}(\theta_k) \neq 0$ en cada iteración. La convergencia a un **mínimo global** requiere convexidad.

El ciclo completo de entrenamiento



El ciclo completo de entrenamiento

Actualización explícita por capa

Para la red $\hat{f} = W_2 \sigma(W_1 x + \mathbf{b}_1) + b_2$, un paso de gradiente:

$$W_1 \leftarrow W_1 - \eta \frac{\partial \mathcal{L}}{\partial W_1}, \quad \mathbf{b}_1 \leftarrow \mathbf{b}_1 - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}_1}, \quad W_2 \leftarrow W_2 - \eta \frac{\partial \mathcal{L}}{\partial W_2}, \quad b_2 \leftarrow b_2 - \eta \frac{\partial \mathcal{L}}{\partial b_2}$$

Backprop calcula los cuatro gradientes en una sola pasada hacia atrás.

JAX con `grad` hace esto automáticamente para cualquier arquitectura.

Parte 4

Backpropagation

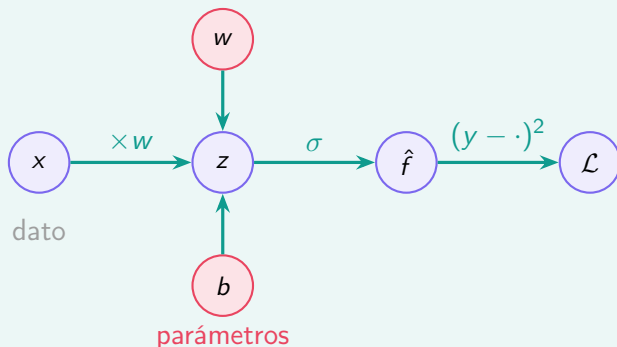
= autodiff en modo reverso sobre el grafo de la red

Regla de la cadena a través de la red

Recordemos: para la red de 1 neurona,

$$z = wx + b, \quad \hat{f} = \sigma(z), \quad \mathcal{L} = (y - \hat{f})^2$$

Grafo computacional



Backprop: pasada hacia adelante y hacia atrás

Forward pass (valores)

$$z = wx + b$$

$$\hat{f} = \sigma(z)$$

$$\mathcal{L} = (y - \hat{f})^2$$

Guardamos z y $\hat{f}$ para usarlos en el backward.

Backward pass (gradientes)

$$\frac{\partial \mathcal{L}}{\partial \hat{f}} = -2(y - \hat{f})$$

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial \hat{f}} \cdot \sigma'(z)$$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot x$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial z} \cdot 1$$

Conexión con la clase anterior

Esto es **exactamente** autodiff en modo reverso: una pasada forward, una pasada backward, gradiente completo.

¿Por qué no diferencias finitas?

Método	Exacto	Costo	Escala a p parámetros
Diferencias finitas	No	$O(p)$ evaluaciones	p veces más lento
Simbólico	Sí	Explosión combinatoria	No factible
Autodiff (backprop)	Sí	$O(1)$ pasadas	Independiente de p

Por qué el gradiente exacto importa

Una red típica tiene $p \sim 10^6$ parámetros.

Con diferencias finitas necesitaríamos $\sim 10^6$ evaluaciones de $\mathcal{L}$ por paso de gradiente.

Con backprop: **2 evaluaciones** (forward + backward).

Ejercicio

Backprop a mano + verificación en JAX

20 minutos

Ejercicio — Parte A: a mano

Red de 2 neuronas, calcular gradientes a mano

Consideren:

$$h_1 = \sigma(w_1x + b_1), \quad h_2 = \sigma(w_2x + b_2), \quad \hat{f} = v_1h_1 + v_2h_2$$

$$\mathcal{L} = (y - \hat{f})^2, \quad \sigma(z) = \tanh(z)$$

Con $x = 1$, $y = 0,5$, $w_1 = 0,3$, $b_1 = 0,1$, $w_2 = -0,2$, $b_2 = 0,4$, $v_1 = 0,6$, $v_2 = 0,8$.

(a) Hagan el forward pass: calculen h_1 , h_2 , $\hat{f}$, $\mathcal{L}$.

(b) Hagan el backward pass: calculen $\partial\mathcal{L}/\partial v_1$, $\partial\mathcal{L}/\partial v_2$, $\partial\mathcal{L}/\partial w_1$, $\partial\mathcal{L}/\partial b_1$.

Ejercicio — Parte B: verificación en JAX

```
import jax.numpy as jnp
from jax import grad, jit
import jax

def red(params, x, y):
    w1, b1, w2, b2, v1, v2 = params
    h1 = jnp.tanh(w1 * x + b1)
    h2 = jnp.tanh(w2 * x + b2)
    f_hat = v1 * h1 + v2 * h2
    return (y - f_hat) ** 2

params0 = jnp.array([0.3, 0.1, -0.2, 0.4, 0.6, 0.8])
x, y = 1.0, 0.5

# Verificar: coincide con el calculo a mano?
gradientes = grad(red)(params0, x, y)
print(gradientes)

# Un paso de descenso de gradiente con eta = 0.1
eta = 0.1
params1 = params0 - eta * gradientes
print("Loss antes:", red(params0, x, y))
print("Loss despues:", red(params1, x, y))
```

`grad(red)` calcula $\nabla_{\theta}\mathcal{L}$ usando backprop exactamente como lo hicieron a mano.

Ejercicio — Parte C: entrenamiento

```
import matplotlib.pyplot as plt

# Funcion objetivo: f(x) = sin(2*pi*x)
key = jax.random.PRNGKey(42)
xs = jnp.linspace(0, 1, 50)
ys = jnp.sin(2 * jnp.pi * xs)

def loss_total(params):
    return jnp.mean(jax.vmap(lambda x, y: red(params, x, y))(xs, ys))

grad_loss = jit(grad(loss_total))

# Descenso de gradiente
params = params0
eta = 0.05
historico = []
for _ in range(500):
    params = params - eta * grad_loss(params)
    historico.append(float(loss_total(params))) # guardar

plt.semilogy(historico)
plt.xlabel("iteracion"); plt.ylabel("L(theta)"); plt.title("Convergencia")
plt.show()
```

¿La red converge? ¿Qué pasa si aumentan m (más neuronas)?

Recapitulando

Red neuronal

$\hat{f}(x; \theta)$ con bases aprendibles — generaliza todo lo anterior

Loss function

$\mathcal{L}(\theta)$ mide el error — es la integral que minimizamos

Backpropagation

autodiff modo reverso sobre el grafo de la red — exacto, $O(1)$ pasadas